

HOW TO EXPLAIN YOUR 3 PROJECTS

Complete spoken scripts + every follow-up question you might be asked

#	Project	Tech Stack	Key Concept
1	Lakehouse Pipeline (Medallion Architecture)	Databricks, Delta Lake, PySpark, SCD Type 2	Bronze→Silver→Gold Data quality + SCD
2	Orchestrated ETL Pipeline (Apache Airflow + PostgreSQL)	Airflow, PyArrow, PostgreSQL, Docker	4-stage DAG 3M+ NYC taxi records
3	Real-Time Streaming Pipeline (Kafka + Spark Streaming)	Kafka, PySpark Structured Streaming, Docker Compose	Live order events Windowed aggregations

This document gives you complete, conversational explanations of all three of your projects. Each project section contains: a full walk-through script (what to say), the architecture diagram in words, every possible follow-up question with a ready answer, and tips on what to emphasize at Visa.

THE FULL STORY — What this project is and why you built it

This is your most impressive project technically. It demonstrates modern data engineering at enterprise scale. The goal was to build a complete, production-grade data pipeline using the Medallion Architecture — the same architecture used by Databricks, Netflix, and Uber — that processes raw e-commerce data and organizes it into clean, analytics-ready Gold layer tables.

■ When introducing this project, always start with: WHY (problem it solves), THEN WHAT (what you built), THEN HOW (technical details). Never jump straight to technical jargon.

■ OPENING EXPLANATION — Say this when asked 'Tell me about your project'

■ SPEAK THIS

This project is an end-to-end data lakehouse pipeline built on Databricks using the Medallion Architecture — a design pattern that organizes data into three progressive layers: Bronze, Silver, and Gold. The problem it solves is common in real-world data engineering: raw e-commerce data coming from multiple sources is messy — it has nulls, duplicates, invalid values, and late-arriving records. You can't directly build analytics on top of it. The Medallion Architecture solves this by creating a structured progression: raw data lands in Bronze, gets cleaned and validated in Silver, and finally gets modeled into an analytics-ready star schema in Gold. I processed over 100,000 e-commerce records — orders, customers, and products — through this pipeline. The key technical achievement was implementing SCD Type 2 for the customer dimension, which means we track the full history of customer changes over time — like address or email updates — rather than just overwriting the old data.

■ ARCHITECTURE WALKTHROUGH — Say this when asked 'Walk me through the architecture'

■ SPEAK THIS

Let me walk you through each layer: **BRONZE LAYER** — This is the raw landing zone. I ingest CSV files into Delta tables with append-only writes. I add two audit columns to every record: `ingestion_timestamp` (when did we load it) and `source_file_name` (which file did it come from). I also enforce the schema here — if the incoming file has a missing column or wrong data type, the write fails with a clear error. No silent corruption. Bronze is immutable — we never modify or delete Bronze records. It's our source of truth for replaying if anything goes wrong downstream. **SILVER LAYER** — This is the validation and cleaning layer. I apply a 4-rule data quality framework: Rule 1 — Null check: `order_id` and `customer_id` cannot be null. Rule 2 — Domain validation: `order_status` must be one of a valid set (pending, shipped, delivered, cancelled). Rule 3 — Future-date filter: `order_date` cannot be in the future. Rule 4 — Deduplication: remove any rows with the same `order_id`. Records that fail any rule go into a separate quarantine table with the error reason — they don't just disappear. Silver data is clean, deduplicated, and trusted. **GOLD LAYER** — This is the analytics-ready layer modeled as a star schema. The center is a `fact_orders` table. Around it are dimension tables: `dim_customer`, `dim_product`, and `dim_date`. `dim_customer` uses SCD Type 2 to track historical changes. This layer is what a BI tool like Tableau or Metabase would connect to.

■ SCD TYPE 2 — Say this when asked 'How did you implement SCD Type 2?'

■ SPEAK THIS

SCD Type 2 — Slowly Changing Dimension Type 2 — is a data warehousing technique for tracking the full history of changes to a dimension. Instead of overwriting old data, we insert a new row and mark the old one as expired. Here's exactly how I implemented it in Delta Lake: Step 1 — Change Detection: For every incoming customer record, I compute a SHA-256 hash of all non-key columns — things like email, city, phone_number. I call this the `change_hash`. Step 2 — Compare with current: I join the incoming records with the existing `dim_customer` table where `is_current = True`. If the hash is different, the customer's data has changed. Step 3 — Delta MERGE: I run a two-step operation using Delta Lake's MERGE command. First, for changed customers: UPDATE the existing row — set `is_current = False`, `effective_end_date = today's date`. Second: INSERT the new version with `is_current = True`, `effective_start_date = today`, `effective_end_date = NULL` (meaning still active). The result is that you can query: 'What was this customer's address in March 2024?' by filtering on `effective_start_date` and `effective_end_date`. Full history, zero data loss.

ALL FOLLOW-UP QUESTIONS & ANSWERS

■ Why did you choose Delta Lake over plain Parquet or a traditional database?

■ SPEAK THIS

Plain Parquet is just a file format — it has no ACID guarantees. If a Spark job fails mid-write, you can end up with partial or corrupted files, and you'd have no way to know. Delta Lake solves this with a transaction log — every write is either fully committed or fully rolled back. It also gives us UPDATE, DELETE, and MERGE operations which plain Parquet doesn't support. And critically for SCD Type 2, we need MERGE — which is impossible with raw Parquet. A traditional database like PostgreSQL doesn't scale for 100K+ records with Spark-level processing. Delta Lake gives us the best of both worlds — data lake scale with database reliability.

■ What is the transaction log in Delta Lake? How does it work?

■ SPEAK THIS

Every Delta table has a hidden folder called `_delta_log`. Every write operation — whether it's an INSERT, UPDATE, DELETE, or MERGE — creates a new JSON file in this log recording what changed: which files were added, which were removed, and the schema. When you read a Delta table, Spark first reads the transaction log to know the latest state of the table, then reads only the relevant Parquet files. This is how ACID is achieved. It's also how Time Travel works — by reading an older version of the transaction log, you can reconstruct the table at any point in history.

■ Why hash instead of comparing columns one by one for change detection?

■ SPEAK THIS

Two reasons. First — maintainability. If the customer table has 20 columns and I compare each one individually, I write 20 conditions in my MERGE logic. If someone adds a column next month, I have to update the comparison code. With a hash, I hash all non-key columns into one value — if anything changes, the hash changes. Zero code change needed when schema evolves. Second — performance. One hash comparison per row is faster than 20 column comparisons per row at scale. The only trade-off is a theoretical hash collision risk, but with SHA-256 that probability is astronomically small — effectively impossible in practice.

■ What happens to records that fail data quality checks?

■ SPEAK THIS

They go to a quarantine table — not just dropped. Each quarantined record has the original data plus an `error_reason` column (e.g., `'null_order_id'`, `'invalid_status'`, `'future_order_date'`) and a `quarantine_timestamp`. This is important for two reasons: first, it gives the data engineering team visibility into data quality issues from the source — if 500 records have `'null_order_id'`, that's a source system bug that needs fixing. Second, it gives us the ability to replay those records after fixing the source issue, because we haven't lost them.

■ What is schema enforcement in Delta Lake?

■ SPEAK THIS

Schema enforcement means Delta Lake will reject any write where the incoming DataFrame doesn't exactly match the table's registered schema. So if the source sends an extra column, or a column has the wrong data type — say a string instead of a date — the write fails with a clear error message rather than silently writing corrupt data. This is critical in production because source systems sometimes change their schema without warning. Schema evolution is the partner feature — if you intentionally want to add a new column, you set `mergeSchema=True` and Delta will update the schema without rewriting all existing data.

■ What is the difference between overwrite and append mode in Delta?

■ SPEAK THIS

Append mode adds new rows without touching existing data — used for Bronze incremental loads. Overwrite mode replaces all data in the table — used when re-processing a full dataset. In my Bronze layer I use append-only writes because Bronze should be immutable — a full audit trail of everything ever ingested. For Silver and Gold I use MERGE (upsert) which is more precise — only touch rows that changed.

■ What is Z-ordering in Delta Lake?

■ SPEAK THIS

Z-ordering is a data layout optimization. `OPTIMIZE ... ZORDER BY (customer_id, order_date)` colocates rows with similar `customer_id` and `order_date` values in the same Parquet files. When a query filters on those columns, Delta can skip entire files that don't contain the relevant values — this is called data skipping. The result is dramatically fewer files read for analytical queries. Think of it like indexing in a database, but for file-based storage.

■ What is a star schema? Why did you use it for Gold layer?

■ SPEAK THIS

A star schema has a central fact table containing measurable events — in my case, `fact_orders` which has one row per order with columns like `order_id`, `customer_key`, `product_key`, `date_key`, `order_amount`, `quantity`. Surrounding it are dimension tables — `dim_customer`, `dim_product`, `dim_date` — which contain descriptive attributes. You join fact to dimension using surrogate keys. I chose star schema for Gold because it's optimized for analytical queries — BI tools like Tableau generate very fast queries against star schemas because joins are simple and predictable. Snowflake schema would normalize the dimensions further but add more joins — overkill for this use case.

■ If you had to make this pipeline production-ready, what would you add?

■ SPEAK THIS

A few things: First — orchestration with Apache Airflow to schedule and monitor the pipeline with retry logic and alerting. Second — data lineage tracking with Unity Catalog or Apache Atlas, so we can trace which source records produced which Gold layer rows. Third — automated data quality monitoring with Great Expectations or dbt tests that alert when quality metrics drop below thresholds. Fourth — CI/CD for notebook/pipeline code with Git integration. Fifth — column-level access control on the Gold layer for PII columns using Unity Catalog row filters.

THE FULL STORY — What this project is and why you built it

This project demonstrates your ability to build a production-grade, orchestrated batch ETL pipeline — exactly the kind of workflow that runs in enterprise data teams every day. The goal was to process 3 million+ NYC TLC (Taxi and Limousine Commission) taxi trip records reliably, with proper validation, retry logic, and full containerization.

■ This is your best project for demonstrating Airflow knowledge and production engineering mindset. Emphasize: orchestration, data quality gate, idempotency, Dockerization. These are exactly the words Visa data engineers use every day.

■ OPENING EXPLANATION

■ SPEAK THIS

This project is a production-grade ETL pipeline that processes over 3 million NYC taxi trip records from the NYC TLC (Taxi and Limousine Commission) open dataset. The pipeline is orchestrated by Apache Airflow and loads validated, transformed data into a PostgreSQL data warehouse. The key engineering challenge wasn't just moving data — it was doing it reliably. Real-world ETL pipelines fail: source files are malformed, network connections drop, databases have transient errors. This pipeline is built to handle all of that gracefully — with automatic retries, data validation gates that block bad data before it reaches the warehouse, and full containerization so it runs identically on any machine.

■ ARCHITECTURE WALKTHROUGH

■ SPEAK THIS

The pipeline is a 4-stage DAG in Apache Airflow — DAG stands for Directed Acyclic Graph, which is just a fancy term for a workflow where tasks run in a defined order with no loops. STAGE 1 — EXTRACT: The first task reads the NYC TLC Parquet file from the source. Here's an important detail: I use PyArrow to read the file's metadata — the schema and row count — WITHOUT loading the full 3 million rows into memory. This is called schema-aware ingestion. It lets me validate that the file is structurally correct before doing any heavy processing. If the schema is wrong, we fail fast without wasting compute. STAGE 2 — VALIDATE: A 4-check quality gate runs before any transformation. Check 1: Schema validation — column names and types match what we expect. Check 2: Null check — trip_id, pickup_datetime, dropoff_datetime cannot be null. Check 3: Business logic validation — trip_distance must be positive, fare_amount must be non-negative. Check 4: Row count sanity — if the file has less than 1000 rows, something is wrong with the extract. If any check fails, the DAG stops here — the bad data never reaches the warehouse. STAGE 3 — TRANSFORM: Apply business logic — rename columns to match warehouse naming conventions, calculate derived fields like trip_duration_minutes, cast data types, handle edge cases like trips with zero distance. STAGE 4 — LOAD: Batch insert into PostgreSQL using the COPY command — which is 10–100x faster than row-by-row INSERT statements for millions of records. Data is loaded in chunks of 100K rows to avoid memory overflow.

ALL FOLLOW-UP QUESTIONS & ANSWERS

■ What is Apache Airflow? Why did you use it instead of just a cron job?

■ SPEAK THIS

Apache Airflow is a workflow orchestration platform — you define pipelines as Python code (DAGs), and Airflow handles scheduling, monitoring, retry logic, dependency management, and alerting. A cron job would run the script but gives you nothing else: no visibility into whether it succeeded, no automatic retries on failure, no dependency tracking between steps, no web UI to monitor history. With Airflow, I get all of that out of the box. If the Load task fails, Airflow retries it 3 times with a 5-minute delay. If it still fails, I get an email alert. I can also see the full run history, re-run individual failed tasks, and backfill historical dates — none of which is possible with cron.

■ What is XCom in Airflow? How did you use it?

■ SPEAK THIS

XCom stands for Cross-Communication — it's Airflow's mechanism for passing small pieces of data between tasks. When a task finishes, it can push a value to XCom using xcom_push. A downstream task can then retrieve that value using xcom_pull. I used XCom to pass the file path from the Extract task to the Validate task, and to pass validation metadata — number of records validated, number rejected — from Validate to Load for logging purposes. Important limitation: XCom stores data in the Airflow metadata database (SQLite or PostgreSQL), so it's only suitable for small values like file paths or counts — never for the actual data itself.

■ What is PyArrow and why use it for metadata reading?

■ SPEAK THIS

PyArrow is a Python library implementing the Apache Arrow columnar memory format. For Parquet files specifically, PyArrow can read the file footer metadata — which contains the schema, row group statistics, column names, and types — without loading the actual row data. I used `parquet.read_schema()` for this. The benefit is significant: a 500MB Parquet file with 3 million rows might take 30 seconds to fully load into memory. Reading just the schema takes milliseconds. For a validation step that might reject the file anyway, it's wasteful to load the full data first. Fail fast, fail cheap.

■ What is idempotency and why is it important for ETL pipelines?

■ SPEAK THIS

Idempotency means running the same pipeline twice produces the same result as running it once — no duplicate data, no partial overwrites. It's critical because ETL pipelines fail and get retried all the time. If my Load task inserts 1 million rows and then crashes, and Airflow retries it, a non-idempotent pipeline would insert those rows a second time — you'd have 2 million rows. I made my pipeline idempotent by using `INSERT ... ON CONFLICT DO UPDATE` (upsert) in PostgreSQL, and by tracking processed file names in a control table — if the file was already loaded successfully, we skip it. Re-running the full DAG is always safe.

■ Why Docker? How did you set up Airflow in Docker?

■ SPEAK THIS

Docker ensures the pipeline runs identically on any machine — dev laptop, CI server, production VM. Without Docker, you deal with 'it works on my machine' problems: different Python versions, different library versions, OS-specific behavior. I used Docker Compose to define three services: `airflow-webserver`, `airflow-scheduler`, and `postgres`. They share a network so Airflow can reach PostgreSQL by service name. The DAGs folder is mounted as a Docker volume so I can edit DAG files locally and they appear in Airflow immediately without rebuilding the container. The stack starts with a single command: `docker-compose up`. That's the definition of a reproducible environment.

■ How did you handle 3 million records without running out of memory?

■ SPEAK THIS

Two techniques. First — chunked loading: instead of loading all 3 million rows at once, I read the Parquet file in chunks of 100,000 rows using PyArrow's batch reading API. Each chunk is processed and loaded separately, so memory usage stays constant regardless of file size. Second — using PostgreSQL `COPY` instead of `INSERT`: the `COPY` command streams data directly from a file/buffer into PostgreSQL without building individual SQL statements for each row. It's the fastest way to bulk-load data into PostgreSQL — benchmarks show 10–50x faster than `executemany()` `INSERT`.

■ What would happen if the PostgreSQL database was unavailable during the load?

■ SPEAK THIS

The Load task would fail. Airflow would catch the exception and, because I've configured `retries=3` with `retry_delay=timedelta(minutes=5)`, it would wait 5 minutes and try again — three times total. If all retries fail, the task is marked as FAILED and an email alert is sent to the pipeline owner. The partial load is safely handled by the idempotent upsert logic — when the pipeline eventually succeeds, it only inserts records that weren't already loaded. No duplicates.

■ What is the difference between a task, a DAG, and a DAG run in Airflow?

■ SPEAK THIS

A DAG (Directed Acyclic Graph) is the definition of a workflow — it describes which tasks exist and in what order they run. Think of it as the blueprint. A Task is a single unit of work within a DAG — in my case: Extract, Validate, Transform, Load. Each task is implemented as a Python function decorated with `@task`. A DAG Run is a specific execution instance of a DAG — if my DAG is scheduled daily, each day's execution is a separate DAG Run with its own `execution_date`, status, and logs. You can have multiple DAG Runs running concurrently if configured.

■ How would you scale this pipeline if data volume grew to 100 million records?

■ SPEAK THIS

At 100 million records, a single-machine Airflow + PostgreSQL setup would struggle. I'd migrate to: Spark for distributed processing (replace pandas with PySpark), move storage from PostgreSQL to a columnar data warehouse like BigQuery, Redshift, or Databricks, use Airflow to orchestrate Spark jobs instead of Python scripts, add partitioning to the target table by date so queries don't scan all 100M rows, and implement incremental loading — only process new records since last run rather than full reload. The Airflow orchestration layer itself scales horizontally with CeleryExecutor or KubernetesExecutor.

PROJECT 3

Real-Time Streaming Pipeline

Kafka · PySpark
Structured Streaming
· Docker

THE FULL STORY — What this project is and why you built it

This project demonstrates real-time data engineering — processing events as they happen rather than in batches. It's the most modern and in-demand skill for data engineers at companies like Visa, which processes 65,000+ transactions per second and needs sub-second fraud detection. You built an end-to-end streaming pipeline from event generation to live aggregation.

■ This is your most valuable project for Visa because Visa's core business IS real-time streaming — every payment authorization is a streaming event. Connect this project to Visa's use case.

■ OPENING EXPLANATION

■ SPEAK THIS

This project is an end-to-end real-time data pipeline built with Apache Kafka and PySpark Structured Streaming. The use case is processing live order events from an e-commerce platform — the same pattern used in payment processing, fraud detection, and real-time inventory management. I built two components: a Kafka producer that continuously generates realistic order events (order_id, product, region, amount, timestamp) and publishes them to a Kafka topic, and a PySpark Structured Streaming consumer that reads from Kafka in real time and performs multi-dimensional aggregations — total sales by product and total sales by region — updating the results every few seconds as new events arrive. The entire stack — Kafka, Zookeeper, and Spark — runs in Docker Compose, so it's fully reproducible with one command.

■ ARCHITECTURE WALKTHROUGH

■ SPEAK THIS

Let me walk through the data flow: **PRODUCER:** A Python script uses the kafka-python library to continuously generate order events. Each event is a JSON object: {order_id, product_name, region, amount, event_timestamp}. The producer uses round-robin partitioning across the Kafka topic's partitions — this distributes load evenly. It publishes approximately 10–20 events per second. **KAFKA TOPIC:** The 'orders' topic has 3 partitions and a replication factor of 1 (single broker setup for local dev). In production, replication factor would be 3 for fault tolerance. Kafka retains messages for 7 days by default — meaning if the consumer goes down, it can replay all missed events from the last committed offset when it comes back up. **PYSPARK CONSUMER:** A PySpark Structured Streaming job reads from the Kafka topic using readStream. Kafka delivers raw bytes — I deserialize the JSON payload into a structured DataFrame with schema. Then I perform two windowed aggregations: Aggregation 1: Total sales amount per product in 1-minute tumbling windows. Aggregation 2: Total sales amount per region in 1-minute tumbling windows. The output goes to the console sink for demo purposes — in production this would write to a Delta Lake table or back to another Kafka topic for downstream consumption. **MONITORING:** Spark UI on port 4040 shows live streaming metrics — input rows per second, batch duration, state store size. I included screenshots of these in the repository as execution proof.

ALL FOLLOW-UP QUESTIONS & ANSWERS

■ What is Apache Kafka? Explain the key concepts.

■ SPEAK THIS

Kafka is a distributed event streaming platform — think of it as a highly durable, ordered message queue that multiple producers can write to and multiple consumers can read from, independently and at their own speed. Key concepts: Topic: A named category of events — like a database table but for streams. My topic is 'orders'. Partition: A topic is split into partitions for parallelism. Each partition is an ordered, immutable sequence of messages. More partitions = more parallel consumers. Producer: Writes messages to a topic. Can choose partition based on key or round-robin. Consumer: Reads messages from a topic. Tracks its position with an offset. Consumer Group: Multiple consumers sharing work — each partition is assigned to one consumer in a group. Broker: A Kafka server that stores partitions. Offset: The position of a consumer within a partition. Kafka remembers where you left off.

■ What is Spark Structured Streaming? How is it different from batch processing?

■ SPEAK THIS

Spark Structured Streaming is Spark's stream processing engine that treats a live data stream as an unbounded table — a table that keeps getting new rows as events arrive. In batch processing, you load a fixed dataset, run transformations, write output, done. In Structured Streaming, you define a query once and Spark continuously runs it as new data arrives — processing micro-batches (every few seconds) or each individual event. The beautiful thing is the API is almost identical to batch. I use the same `select()`, `groupBy()`, `agg()` functions — Spark handles the incremental computation automatically. For aggregations, Spark maintains state across micro-batches — it remembers partial aggregates and updates them as new events arrive, without reprocessing old data.

■ What is the difference between a tumbling window and a sliding window?

■ SPEAK THIS

A tumbling window is a fixed-size, non-overlapping time bucket. Example: 1-minute tumbling windows create buckets [00:00–01:00], [01:00–02:00], [02:00–03:00]. Each event belongs to exactly one window. Use tumbling for disjoint period summaries like 'sales per minute' or 'transactions per hour'. A sliding window is defined by two parameters: window size and slide interval. Example: window=5min, slide=1min creates overlapping buckets: [00:00–05:00], [01:00–06:00], [02:00–07:00]. Each event can appear in multiple windows. Use sliding for moving averages like '5-minute rolling average of transaction amount'. I used tumbling windows in my project because I wanted clean, non-overlapping 1-minute summaries of order volume per product and region.

■ What is an offset in Kafka? Why does it matter for fault tolerance?

■ SPEAK THIS

An offset is a sequential integer ID that uniquely identifies each message within a partition. Message 0, message 1, message 2... The consumer tracks which offset it has processed up to. This is what makes Kafka fault-tolerant. If my Spark streaming job crashes while processing offset 1000, it can restart and resume from offset 1000 — not from the beginning, not from wherever new messages are. Structured Streaming automatically checkpoints the last committed offset to a fault-tolerant store (HDFS, S3, or local filesystem) after each micro-batch. This means exactly-once or at-least-once processing semantics — no data loss.

■ What is checkpointing in Spark Structured Streaming?

■ SPEAK THIS

Checkpointing is how Spark Structured Streaming achieves fault tolerance. After every micro-batch completes successfully, Spark writes two things to the checkpoint location: the offset range processed in this batch (so we know where to resume if we crash), and the intermediate aggregation state (so we don't lose partially computed aggregates). I set `checkpointLocation` to a local directory in my Docker setup — in production, this would be an S3 or HDFS path accessible to all Spark executors. Without checkpointing, a Spark restart would either reprocess all data or lose all aggregation state built up so far.

■ What is Zookeeper's role in your Kafka setup?

■ SPEAK THIS

In my setup I'm using Kafka 2.x which still requires Zookeeper. Zookeeper manages cluster metadata for Kafka: which broker is the leader for each partition, cluster membership (which brokers are alive), consumer group coordination, and configuration storage for topics. When a broker fails, Zookeeper detects it and triggers leader election for the partitions that broker was leading. In newer Kafka 3.x, Zookeeper has been replaced by KRaft — Kafka's own built-in Raft consensus implementation — removing the operational complexity of managing a separate Zookeeper cluster.

■ What is the output mode in Structured Streaming? Which did you use?

■ SPEAK THIS

Output mode defines what rows are written to the sink in each micro-batch. Append mode: only newly added rows are written — for immutable event streams with no aggregation. Complete mode: the entire result table is written every batch — for small aggregations where you want the full current state each time. Update mode: only rows that changed since the last batch are written — most efficient for aggregations with many keys. I used Update mode for my windowed aggregations because I only want to write the product/region sales totals that changed in the latest batch — not re-write the entire aggregation table every second. This is much more efficient when there are many active windows.

■ How would you connect this to a production fraud detection pipeline like Visa's?

■ SPEAK THIS

At Visa's scale, the pattern would be similar but more sophisticated. Instead of order events, the Kafka topic would carry authorization events — card number (tokenized), merchant_id, amount, location, timestamp. The Spark Structured Streaming job would compute real-time features: transaction velocity per card in last 5 minutes, average transaction amount per card in last 30 minutes, geographic anomaly score (is this location different from usual?). These features would be written to a low-latency feature store like Redis or Apache Pinot, where an ML fraud scoring model reads them for real-time inference. High-confidence fraud alerts would go to a separate Kafka topic consumed by the authorization system. All raw events and scored results would land in Delta Lake for batch analytics and model retraining. That's the Lambda/Kappa architecture — speed layer for real-time + batch layer for accuracy.

■ What are the limitations of your current implementation?

■ SPEAK THIS

My current implementation is a single-node setup designed for learning and demonstration. Limitations and how I'd address them in production: 1. Single Kafka broker — no replication. Fix: 3-broker cluster with replication factor 3. 2. Console sink — data isn't persisted. Fix: write to Delta Lake with APPEND mode. 3. No schema registry — JSON schema can drift silently. Fix: Confluent Schema Registry enforces Avro/Protobuf schemas on every message. 4. No backpressure handling — if Spark processes slower than Kafka produces, lag builds up. Fix: configure `maxOffsetsPerTrigger` to control throughput. 5. No monitoring — no alerts if the consumer falls behind. Fix: track consumer group lag metrics with Kafka Exporter + Grafana.

MASTER TIP — How to Structure Any Project Explanation

■ For ANY project question, use this formula: 1. PROBLEM (10 sec): 'The problem this solves is...' 2. SOLUTION OVERVIEW (20 sec): 'I built a pipeline that...' 3. ARCHITECTURE (60 sec): Walk through each component left to right 4. HARDEST PART (20 sec): 'The most challenging part was... here's how I solved it' 5. RESULT / IMPACT (10 sec): 'The result was... records processed, uptime achieved, improvement made' Never say 'It's just a project I did for learning.' Say: 'This solves a real-world problem and here's why the technical decisions I made matter at production scale.'

You know these projects inside out — you built them. Trust yourself, speak confidently, and connect every answer back to real-world production scenarios. Good luck Annjan! ■